
Modelling and Performance using MATLAB

ECE Department, University of Kufa
Dr. Bahaa Al-Musawi

Outline

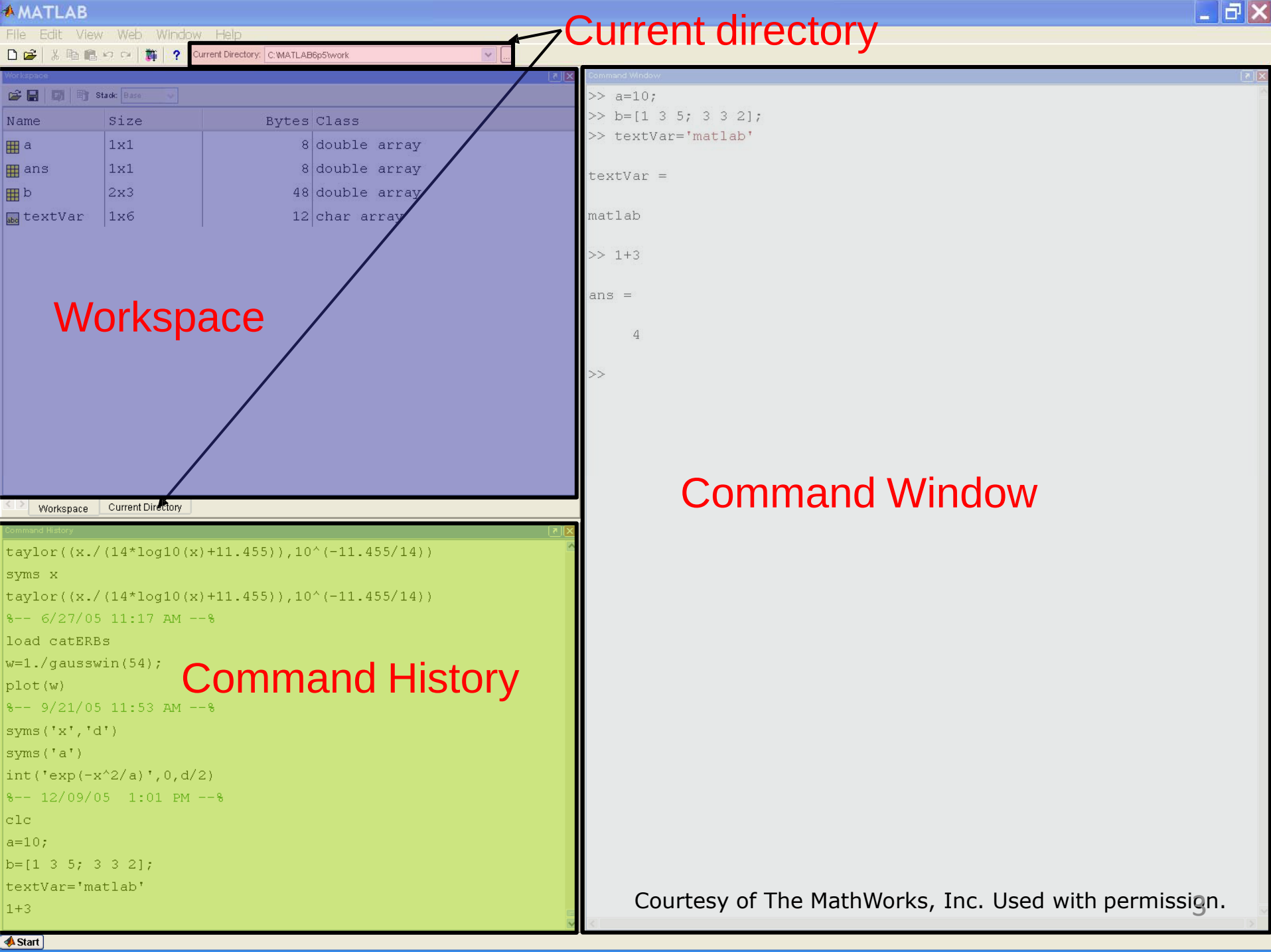
(1) Getting Started

(2) Scripts

(3) Making Variables

(4) Manipulating Variables

(5) Basic Plotting

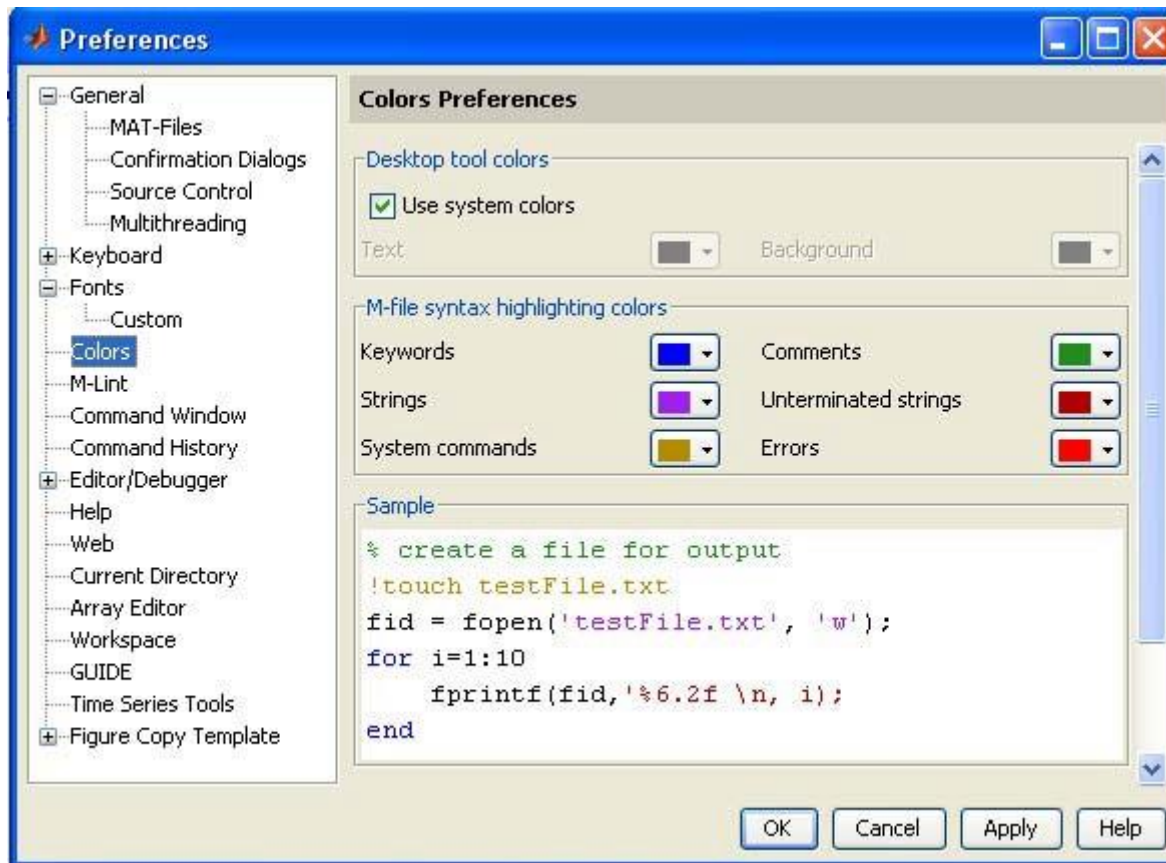


Making Folders

- Use folders to keep your programs organized
- To make a new folder, click the 'Browse' button next to 'Current Directory'
- Click the 'Make New Folder' button, and change the name of the folder. **Do NOT use spaces** in folder names. In the MATLAB folder, make two new folders: `IAPMATLAB\day1`
- Highlight the folder you just made and click 'OK'
- The current directory is now the folder you just created
- To see programs outside the current directory, they should be in the Path. Use File-> Set Path to add folders to the path

Customization

- File --> Preferences
 - Allows you personalize your MATLAB experience



Help/Docs

- `help`
 - ❖ **The most** important function for learning MATLAB on your own
- To get info on how to use a function:
 - » `help sin`
 - ❖ Help lists related functions at the bottom and links to the doc
- To get a nicer version of help with examples and easy-to-read descriptions:
 - » `doc sin`
- To search for a function by specifying keywords:
 - » `doc` + Search tab

Outline

(1) Getting Started

(2) Scripts

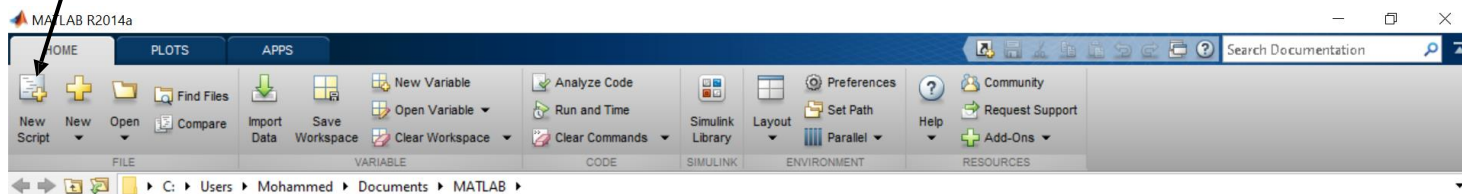
(3) Making Variables

(4) Manipulating Variables

(5) Basic Plotting

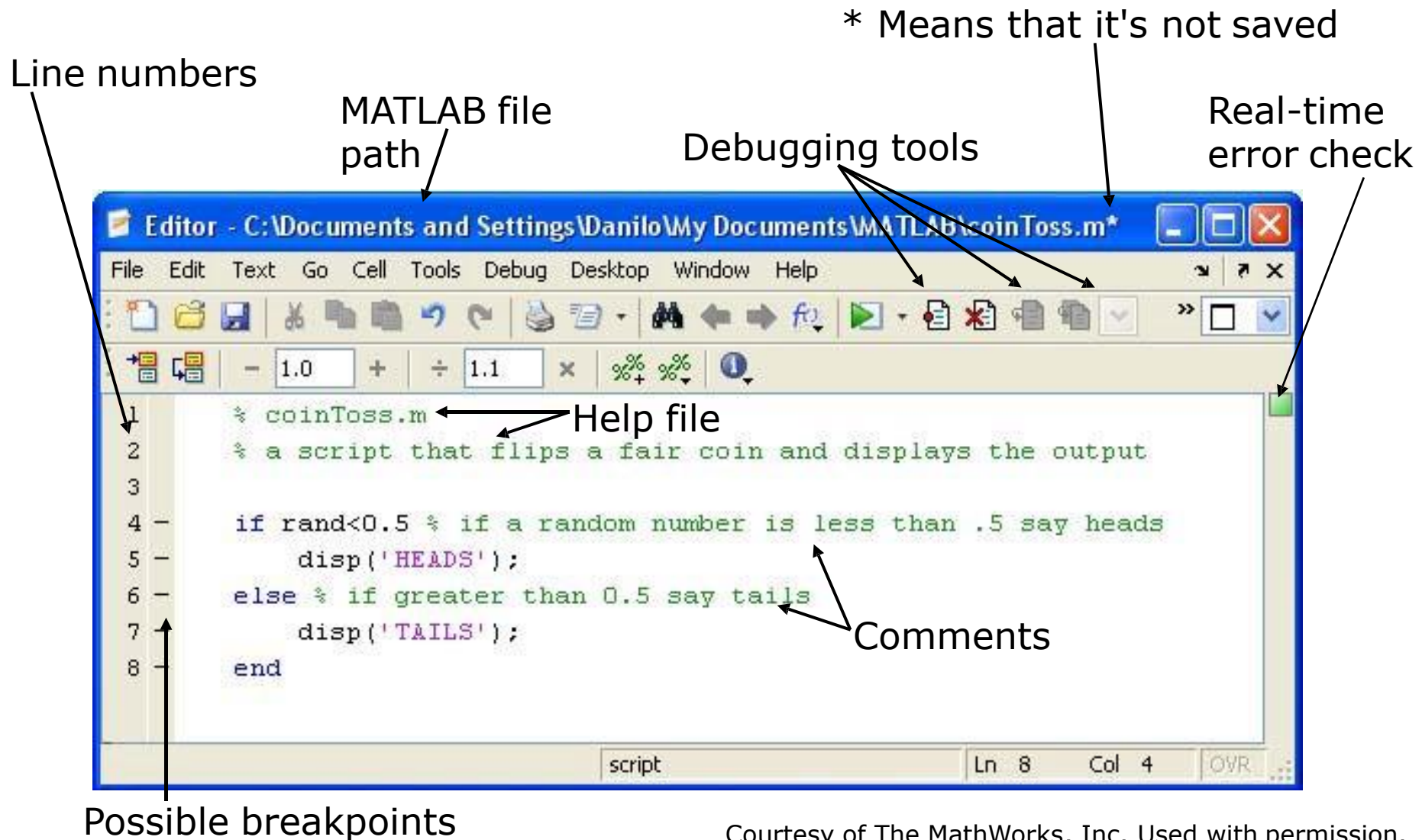
Scripts: Overview

- Scripts are
 - collection of commands executed in sequence
 - written in the MATLAB editor
 - saved as MATLAB files (.m extension)
- To create an MATLAB file from command-line
 - » `edit helloWorld.m`
- or click



Courtesy of The MathWorks, Inc. Used with permission.

Scripts: the Editor



Courtesy of The MathWorks, Inc. Used with permission.

Scripts: Some Notes

- **COMMENT!**

- Anything following a % is seen as a comment
- The first contiguous comment becomes the script's help file
- Comment thoroughly to avoid wasting time later

- Note that scripts are somewhat static, since there is no input and no explicit output
- All variables created and modified in a script exist in the workspace even after it has stopped running

Exercise: Scripts

Make a `helloWorld` script

- When run, the script should display the following text:

Hello World!

I am going to learn MATLAB!

- **Hint:** use `disp` to display strings. Strings are written between single quotes, like `'This is a string'`

Exercise: Scripts

Make a `helloWorld` script

- When run, the script should display the following text:

Hello World!

I am going to learn MATLAB!

- **Hint:** use `disp` to display strings. Strings are written between single quotes, like `'This is a string'`
- Open the editor and save a script as `helloWorld.m`. This is an easy script, containing two lines of code:
 - » `% helloWorld.m`
 - » `% my first hello world program in MATLAB`
 - » `disp('Hello World!');`
 - » `disp('I am going to learn MATLAB!');`

Outline

- (1) Getting Started
- (2) Scripts
- (3) Making Variables**
- (4) Manipulating Variables
- (5) Basic Plotting

Variable Types

- MATLAB is a weakly typed language
 - ❖ No need to initialize variables!
- MATLAB supports various types, the most often used are
 - » `3.84`
 - ❖ 64-bit double (default)
 - » `'a'`
 - ❖ 16-bit char
- Most variables you'll deal with will be vectors or matrices of doubles or chars
- Other types are also supported: complex, symbolic, 16-bit and 8 bit integers, etc. You will be exposed to all these types through the homework

List of Variable Types

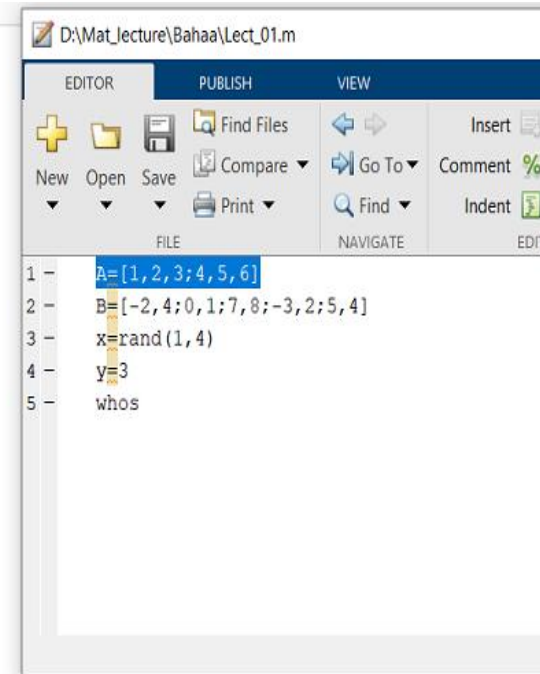
- Numerical Values (or numbers) and integers
- Symbolic variables (no numerical values implied)
- Complex numbers
- Vectors and Matrices (2D and nD)
- Logical variables (true and false)
- Strings (for string textual expressions)
- Mathematical functions and expressions
- Laplace transforms
- Cells and structures

Numerical Values

- The most common variable type, in essence the default is a matrix
 - Include vectors and scalars
- MATLAB denotes these with the classification of “double”

Command Window

```
A =  
    1    2    3  
    4    5    6  
  
B =  
   -2    4  
    0    1  
    7    8  
   -3    2  
    5    4  
  
x =  
    0.0975    0.2785    0.5469    0.9575  
  
y =  
    3  
  
Name      Size      Bytes  Class  Attributes  
A         2x3         48  double  
B         5x2         80  double  
x         1x4         32  double  
y         1x1          8  double
```



Symbolic variables

- Symbolic variables do not contain values and allow the user to write algebraic expressions
 - Supports operation such as differentiation
 - Matrix denotes these with a classification of "sym"

```
Command Window
>> syms w z alpha gama
>> f=alpha*w+z^2*cos(gama)

f =

cos(gama)*z^2 + alpha*w

>> whos w z alpha gama
      Name      Size      Bytes  Class  Attributes
      alpha      1x1         8   sym
      gama      1x1         8   sym
      w         1x1         8   sym
      z         1x1         8   sym

>> diff(f,z)

ans =

2*z*cos(gama)

fx >>
```

Complex numbers

- The default value in a matrix is always a complex number
 - Complex numbers are also captured in the classification of "double"

```
Command Window
>> a=1-j; b=2-2j;c=3+3j;d=4+4j;
>> f=[a b;c d]

f =

    1.0000 - 1.0000i    2.0000 - 2.0000i
    3.0000 + 3.0000i    4.0000 + 4.0000i

>> imaginary_of_f=imag(f)

imaginary_of_f =

    -1    -2
     3     4

>> whos a b c d imaginary_of_f
```

Name	Size	Bytes	Class	Attributes
a	1x1	16	double	complex
b	1x1	16	double	complex
c	1x1	16	double	complex
d	1x1	16	double	complex
imaginary_of_f	2x2	32	double	

Logical variables

- Logical variables are useful tools as these are often used in decision making
 - MATLAB has a special class of this is "logical"

```
Command Window
>> yes=logical(1);
no=logical(0);
yes_or_no=or(yes,no)
yes_and_no=and(yes,no)
not_function=not(no)
whos

yes_or_no =

    logical

     1

yes_and_no =

    logical

     0

not_function =

    logical

     1
```

Name	Size	Bytes	Class	Attributes
no	1x1	1	logical	
not_function	1x1	1	logical	
yes	1x1	1	logical	

Strings

- Sometimes we want text for titles and labels and also where this is more expressive than a number
 - MATLAB denote "test" with the classification of "char"

```
Command Window
```

```
>> whos
```

Name	Size	Bytes	Class	Attributes
ex	1x2	4	char	
num1	1x3	6	char	
num2	1x5	10	char	
num3	1x3	24	double	
str1	1x9	18	char	
str2	1x7	14	char	

```
fx >>
```

```
Command Window
```

```
>> str1='Hello ECE';  
str2='Bye ECE';  
ex=[str1(7),str1(3)]  
ex=[str1(7),str2(3)]  
num1=num2str(200);  
num1  
num2='3 4 5'  
num3=str2num(num2)
```

```
ex =  
  
    'E1'  
  
ex =  
  
    'Ee'  
  
num1 =  
  
    '200'  
  
num2 =  
  
    '3 4 5'  
  
num3 =  
  
    3    4    5
```

```
fx
```

Structures

- Students will often store multiple resources in a single folder
- MATLAB allows you to store multiple related variables under a single variable name, thus making saving and transfer of this information more efficient
 - Matrix denotes these with the classification of “structure”, see ‘help struct’ for more information

Command Window

```
>> a=3;x=[1 2 3];A=[1 2;3 4];st='example';z=1+2j;  
>> All.a=a;  
>> All.x=x;  
>> All.A=A;  
>> All.st=st;  
>> All.z=z;  
>> All
```

All =

struct with fields:

```
a: 3  
x: [1 2 3]  
A: [2x2 double]  
st: 'example'  
z: 1.0000 + 2.0000i
```

```
>> whos All
```

Name	Size	Bytes	Class	Attributes
All	1x1	934	struct	

Cells

- These are conceptual equivalent to structure arrays, but information stored by index position rather than name
 - Matrix denotes these with the classification of "cell"

Pre-allocating Cell Arrays with the cell Function

Creating Cell Arrays with Assignment Statements

Command Window

```
>> A(1,1) = {[1 4 3; 0 5 8; 7 2 9]};  
A(1,2) = {'Anne Smith'};  
A(2,1) = {3+7i};  
A(2,2) = {-pi:pi/10:pi};  
A(3,3)={5};  
A  
  
A =  
  
3x3 cell array  
  
{3x3 double } {'Anne Smith'} {0x0 double}  
{[3.0000 + 7.0000i]} {1x21 double } {0x0 double}  
{0x0 double } {0x0 double } {[ 5]}
```

fx >> |

Command Window

```
>> B=cell(2,3)  
B(1,3)={1:3};  
B  
  
B =  
  
2x3 cell array  
  
{0x0 double} {0x0 double} {0x0 double}  
{0x0 double} {0x0 double} {0x0 double}  
  
B =  
  
2x3 cell array  
  
{0x0 double} {0x0 double} {1x3 double}  
{0x0 double} {0x0 double} {0x0 double}
```

fx >>

Anonymous functions

- A useful shorthand to define mathematical functions of the type $y=f(x)$ without creating a separate m-file
 - MATLAB denotes these with the classification of "function_handle"

Command Window

```
>> g=@(x)x*sin(x)+3
g(2)

g =

function_handle with value:

@(x)x*sin(x)+3

ans =

4.8186

>> g=@(x,y)x*sin(y)+3
g(2,3)

g =

function_handle with value:

@(x,y)x*sin(y)+3

ans =

3.2822
```

Arrays

- Like other programming languages, arrays are an important part of MATLAB
- Two types of arrays

(1) matrix of numbers (either double or complex)

(2) cell array of objects (more advanced data structure)

**MATLAB makes vectors easy!
That's its power!**



Row Vectors

- Row vector: comma or space separated values between brackets

```
» row = [1 2 5.4 -6.6]
```

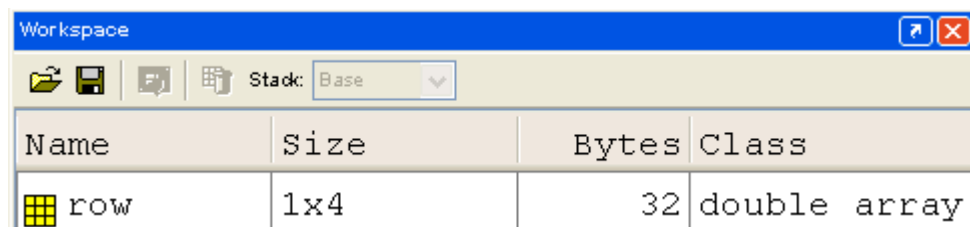
```
» row = [1, 2, 5.4, -6.6];
```

- Command window: `>> row=[1 2 5.4 -6.6]`

```
row =
```

```
1.0000    2.0000    5.4000   -6.6000
```

- Workspace:



The image shows a screenshot of the MATLAB Workspace window. The window has a blue title bar with the text 'Workspace' and standard window controls. Below the title bar is a toolbar with icons for saving, deleting, and refreshing, along with a 'Stack' dropdown menu set to 'Base'. The main area of the window is a table with four columns: 'Name', 'Size', 'Bytes', and 'Class'. There is one row in the table representing the variable 'row', which is a 1x4 double array, occupying 32 bytes.

Name	Size	Bytes	Class
row	1x4	32	double array

Column Vectors

- Column vector: semicolon separated values between brackets

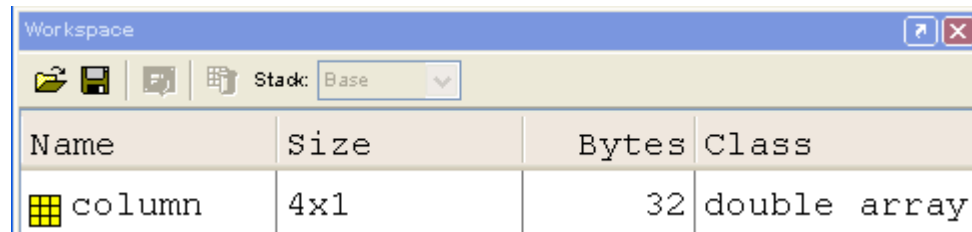
```
» column = [4;2;7;4]
```

- Command window: `>> column=[4;2;7;4]`

```
column =
```

```
4
2
7
4
```

- Workspace:



The image shows a screenshot of the MATLAB Workspace window. The window has a title bar 'Workspace' and a toolbar with icons for saving, deleting, and refreshing. Below the toolbar is a 'Stack' dropdown menu set to 'Base'. The main area is a table with four columns: 'Name', 'Size', 'Bytes', and 'Class'. There is one row in the table representing the variable 'column'.

Name	Size	Bytes	Class
column	4x1	32	double array

size & length

- You can tell the difference between a row and a column vector by:
 - Looking in the workspace
 - Displaying the variable in the command window
 - Using the size function

<pre>>> size(row)</pre>	<pre>>> size(column)</pre>
<pre>ans =</pre>	<pre>ans =</pre>
<pre>1 4</pre>	<pre>4 1</pre>

- To get a vector's length, use the length function

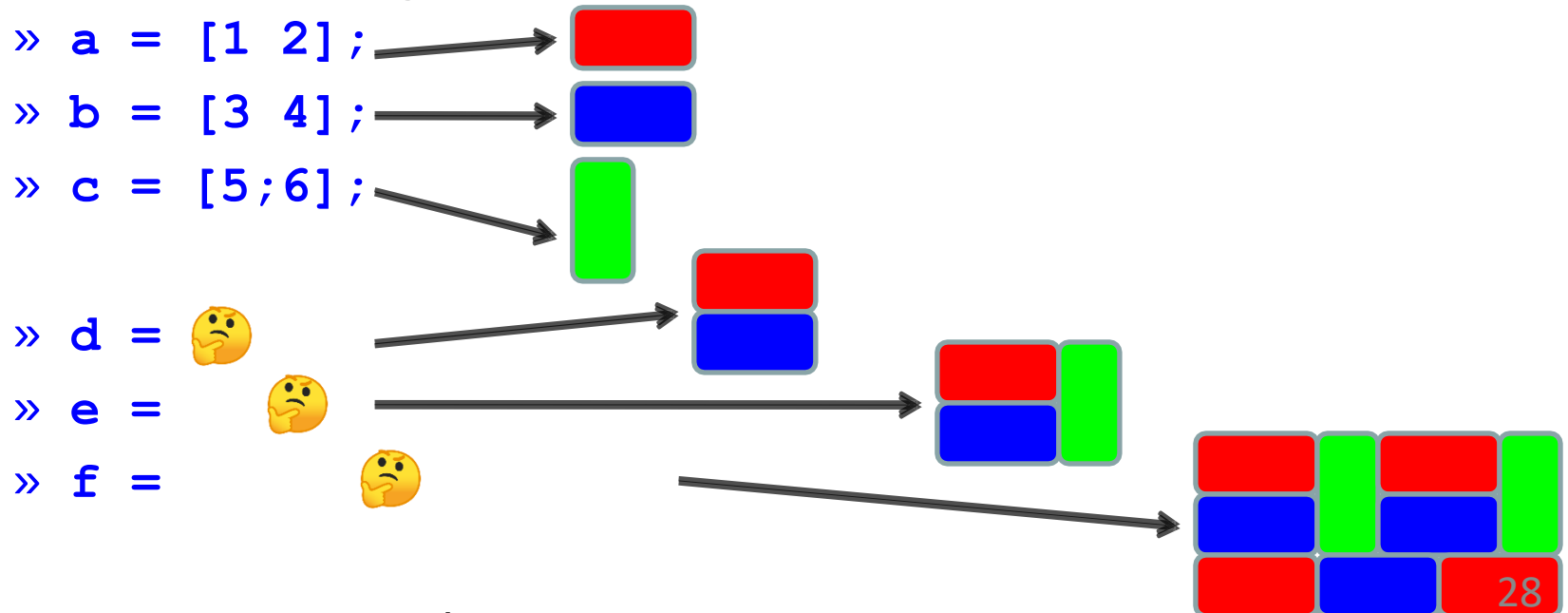
<pre>>> length(row)</pre>	<pre>>> length(column)</pre>
<pre>ans =</pre>	<pre>ans =</pre>
<pre>4</pre>	<pre>4</pre>

Matrices

- Make matrices like vectors

- Element by element
» `a = [1 2; 3 4];` → $a = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

- By concatenating vectors or matrices (dimension matters)



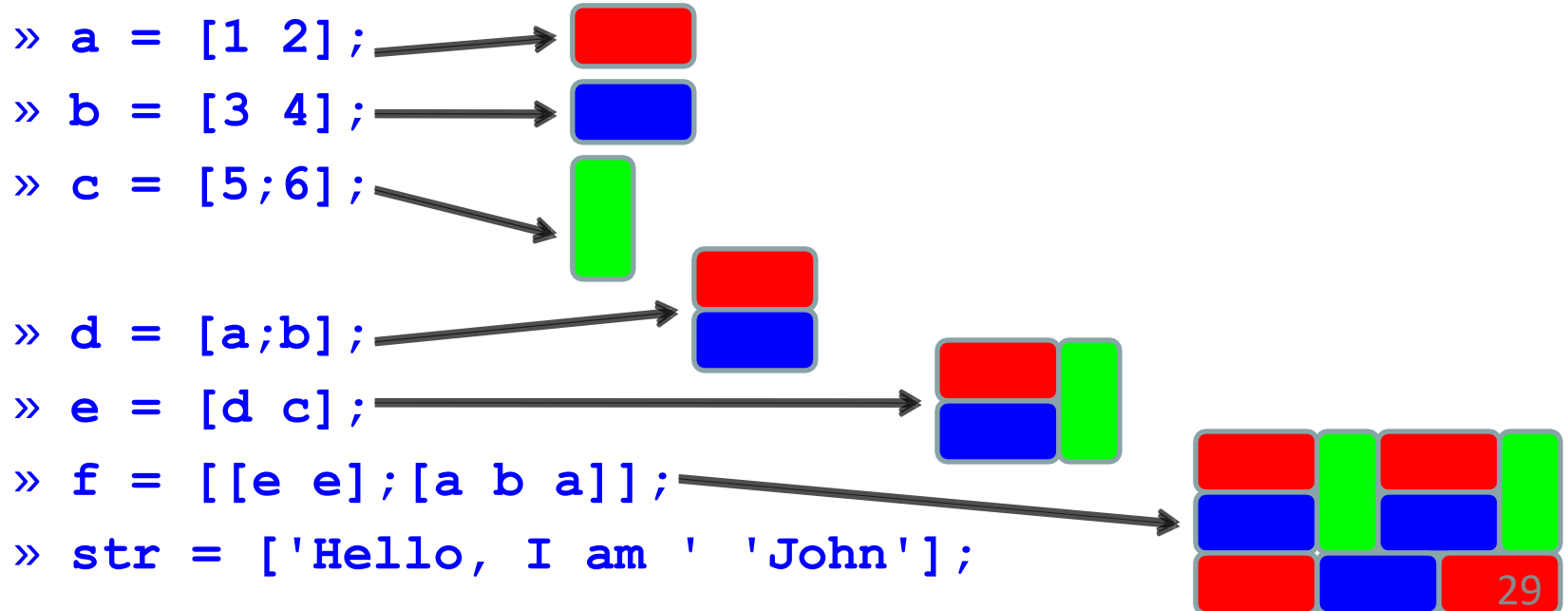
➤ Strings are character vectors

Matrices

- Make matrices like vectors

- Element by element
» `a = [1 2; 3 4];` → $a = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

- By concatenating vectors or matrices (dimension matters)



➤ Strings are character vectors

save/clear/load

- Use **save** to save variables to a file
 - » `save myFile a b`
 - saves variables a and b to the file myfile.mat
 - myfile.mat file is saved in the current directory
 - Default working directory is
 - » `\MATLAB`
 - Make sure you're in the desired folder when saving files. Right now, we should be in:
 - » `MATLAB\IAPMATLAB\day1`
- Use **clear** to remove variables from environment
 - » `clear a b`
 - look at workspace, the variables a and b are gone
- Use **load** to load variable bindings into the environment
 - » `load myFile`
 - look at workspace, the variables a and b are back
- Can do the same for entire environment
 - » `save myenv; clear all; load myenv;`

Exercise: Variables

Get and save the current date and time

- Create a variable `start` using the function `clock`
- What is the size of `start`? Is it a row or column?
- What does `start` contain? See `help clock`
- Convert the vector `start` to a string. Use the function `datestr` and name the new variable `startString`
- Save `start` and `startString` into a mat file named `startTime`

Exercise: Variables

Get and save the current date and time

- Create a variable `start` using the function `clock`
- What is the size of `start`? Is it a row or column?
- What does `start` contain? See `help clock`
- Convert the vector `start` to a string. Use the function `datestr` and name the new variable `startString`
- Save `start` and `startString` into a mat file named `startTime`

```
» help clock
» start=clock;
» size(start)
» help datestr
» startString=datestr(start);
» save startTime start startString
```


Exercise: Variables

Read in and display the current date and time

- In `helloWorld.m`, read in the variables you just saved using `load`
- Display the following text:
I started learning MATLAB on *start date and time*
- **Hint:** use the `disp` command again, and remember that strings are just vectors of characters so you can join two strings by making a row vector with the two strings as sub-vectors.

Exercise: Variables

Read in and display the current date and time

- In helloWorld.m, read in the variables you just saved using `load`
- Display the following text:
I started learning MATLAB on *start date and time*
- **Hint:** use the `disp` command again, and remember that strings are just vectors of characters so you can join two strings by making a row vector with the two strings as sub-vectors.

```
» load startTime
```

```
» disp(['I started learning MATLAB on ' ...  
startString]);
```

Outline

- (1) Getting Started
- (2) Scripts
- (3) Making Variables
- (4) Manipulating Variables**
- (5) Basic Plotting

Basic Scalar Operations

- Arithmetic operations (+, -, *, /)
 - » 7/45
 - » (1+i)*(2+i)
 - » 1 / 0
 - » 0 / 0
- Exponentiation (^)
 - » 4^2
 - » (3+4*j)^2
- Complicated expressions, use parentheses
 - » ((2+3)*3)^0.1
- Multiplication is NOT implicit given parentheses
 - » 3(1+0.7) gives an error
- To clear command window
 - » clc

Built-in Functions

- MATLAB has an **enormous** library of built-in functions
- Call using parentheses – passing parameter to function
 - » `sqrt(2)`
 - » `log(2), log10(0.23)`
 - » `cos(1.2), atan(-.8)`
 - » `exp(2+4*i)`
 - » `round(1.4), floor(3.3), ceil(4.23)`
 - » `angle(i); abs(1+i);`

Transpose

- The transpose operators turns a column vector into a row vector and vice versa
 - » `a = [1 2 3 4+i]`
 - » `transpose(a)`
 - » `a'`
 - » `a.'`
- The `'` gives the Hermitian-transpose, i.e. transposes and conjugates all complex numbers
- For vectors of real numbers `.'` and `'` give same result

Addition and Subtraction

- Addition and subtraction are element-wise; sizes must match (unless one is a scalar):

$$\begin{array}{r} [12 \quad 3 \quad 32 \quad -11] \\ + [2 \quad 11 \quad -30 \quad 32] \\ \hline = [14 \quad 14 \quad 2 \quad 21] \end{array}$$

$$\begin{bmatrix} 12 \\ 1 \\ -10 \\ 0 \end{bmatrix} - \begin{bmatrix} 3 \\ -1 \\ 13 \\ 33 \end{bmatrix} = \begin{bmatrix} 9 \\ 2 \\ -23 \\ -33 \end{bmatrix}$$

- The following would give an error
 - » `c = row + column`
- Use the transpose to make sizes compatible
 - » `c = row' + column`
 - » `c = row + column'`
- Can sum up or multiply elements of vector
 - » `s=sum(row) ;`
 - » `p=prod(row) ;`

Element-Wise Functions

- All the functions that work on scalars also work on vectors
 - » `t = [1 2 3];`
 - » `f = exp(t);`
 - is the same as
 - » `f = [exp(1) exp(2) exp(3)];`
- If in doubt, check a function's help file to see if it handles vectors elementwise
- Operators (`*` `/` `^`) have two modes of operation
 - element-wise
 - standard

Operators: element-wise

- To do element-wise operations, use the dot: `.` (`.*`, `./`, `.^`). BOTH dimensions must match (unless one is scalar)!

» `a=[1 2 3];b=[4;2;1];`

» `a.*b`, `a./b`, `a.^b` → all errors

» `a.*b'`, `a./b'`, `a.^(b')` → all valid

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} .* \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix} = \text{ERROR}$$

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} .* \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4 \\ 3 \end{bmatrix}$$

$$3 \times 1 .* 3 \times 1 = 3 \times 1$$

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix} .* \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$

$$3 \times 3 .* 3 \times 3 = 3 \times 3$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} .^2 = \begin{bmatrix} 1^2 & 2^2 \\ 3^2 & 4^2 \end{bmatrix}$$

Can be any dimension

Operators: standard

- Multiplication can be done in a standard way or element-wise
- Standard multiplication ($*$) is either a dot-product or an outer-product
 - Remember from linear algebra: inner dimensions must MATCH!!
- Standard exponentiation ($^$) can only be done on square matrices or scalars
- Left and right division ($/$ $\backslash$) is same as multiplying by inverse
 - Our recommendation: just multiply by inverse (more on this later)

$$\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} * \begin{bmatrix} 4 \\ 2 \\ 1 \end{bmatrix} = 11$$

$1 \times 3 * 3 \times 1 = 1 \times 1$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} ^2 = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} * \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Must be square to do powers

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \\ 1 & 2 & 3 \\ 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 6 & 9 \\ 6 & 12 & 18 \\ 9 & 18 & 27 \end{bmatrix}$$

$3 \times 3 * 3 \times 3 = 3 \times 3$

Automatic Initialization

- Initialize a vector of **ones**, **zeros**, or **random** numbers
 - » `o=ones(1,10)`
 - row vector with 10 elements, all 1
 - » `z=zeros(23,1)`
 - column vector with 23 elements, all 0
 - » `r=rand(1,45)`
 - row vector with 45 elements (uniform [0,1])
 - » `n=nan(1,69)`
 - row vector of NaNs (useful for representing uninitialized variables)

The general function call is:

```
var=zeros(M,N);
```

Number of rows

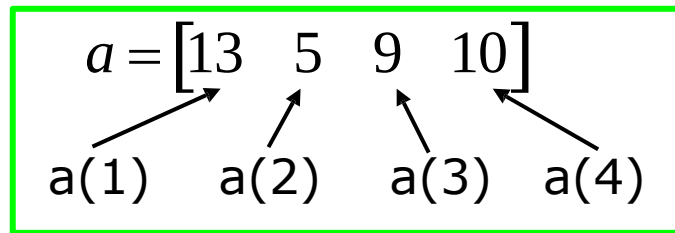
Number of columns

Automatic Initialization

- To initialize a linear vector of values use **linspace**
 - » `a=linspace(0,10,5)`
 - starts at 0, ends at 10 (inclusive), 5 values
- Can also use colon operator (**:**)
 - » `b=0:2:10`
 - starts at 0, increments by 2, and ends at or before 10
 - increment can be decimal or negative
 - » `c=1:5`
 - if increment isn't specified, default is 1
- To initialize logarithmically spaced values use **logspace**
 - similar to **linspace**, but see **help**

Vector Indexing

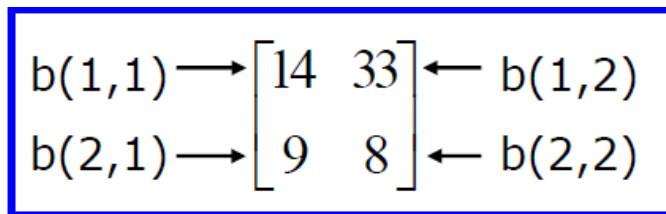
- MATLAB indexing starts with **1**, not **0**
- $a(n)$ returns the n^{th} element

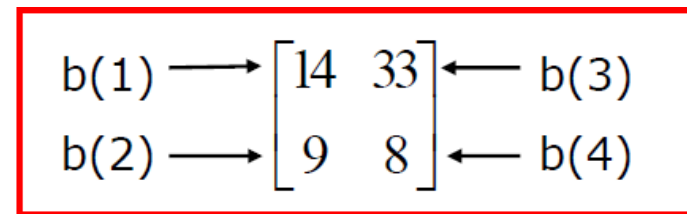


- The index argument can be a vector. In this case, each element is looked up individually, and returned as a vector of the same size as the index vector.
 - » $x = [12 \ 13 \ 5 \ 8];$
 - » $a = x(2:3);$ —————→ $a = [13 \ 5];$
 - » $b = x(1:end-1);$ —————→ $b = [12 \ 13 \ 5];$

Matrix Indexing

- Matrices can be indexed in two ways
 - using **subscripts** (row and column)
 - using linear **indices** (as if matrix is a vector)
- Matrix indexing: **subscripts** or **linear indices**


$$\begin{array}{l} b(1,1) \longrightarrow \begin{bmatrix} 14 & 33 \\ 9 & 8 \end{bmatrix} \longleftarrow b(1,2) \\ b(2,1) \longrightarrow \quad \quad \quad \longleftarrow b(2,2) \end{array}$$


$$\begin{array}{l} b(1) \longrightarrow \begin{bmatrix} 14 & 33 \\ 9 & 8 \end{bmatrix} \longleftarrow b(3) \\ b(2) \longrightarrow \quad \quad \quad \longleftarrow b(4) \end{array}$$

- Picking submatrices
 - » `A = rand(5)` % shorthand for 5x5 matrix
 - » `A(1:3,1:2)` % specify contiguous submatrix
 - » `A([1 5 3], [1 4])` % specify rows and columns

Advanced Indexing 1

- To select rows or columns of a matrix, use the **:**

$$c = \begin{bmatrix} 12 & 5 \\ -2 & 13 \end{bmatrix}$$

» `d=c(1, :)` ;  `d=[12 5]` ;
» `e=c(:, 2)` ;  `e=[5;13]` ;
» `c(2, :)= [3 6]` ; %replaces second row of c

Advanced Indexing 2

- MATLAB contains functions to help you find desired values within a vector or matrix
 - » `vec = [5 3 1 9 7]`
- To get the minimum value and its index:
 - » `[minVal,minInd] = min(vec);`
 - ❖ `max` works the same way
- To find any the indices of specific values or ranges
 - » `ind = find(vec == 9);`
 - » `ind = find(vec > 2 & vec < 6);`
 - `find` expressions can be very complex, more on this later
- To convert between subscripts and indices, use `ind2sub`, and `sub2ind`. Look up `help` to see how to use them.

Outline

- (1) Getting Started
- (2) Scripts
- (3) Making Variables
- (4) Manipulating Variables
- (5) Basic Plotting**

Plotting

- Example
 - » `x=linspace(0,4*pi,10);`
 - » `y=sin(x);`
- Plot values against their index
 - » `plot(y);`
- Usually we want to plot y versus x
 - » `plot(x,y);`

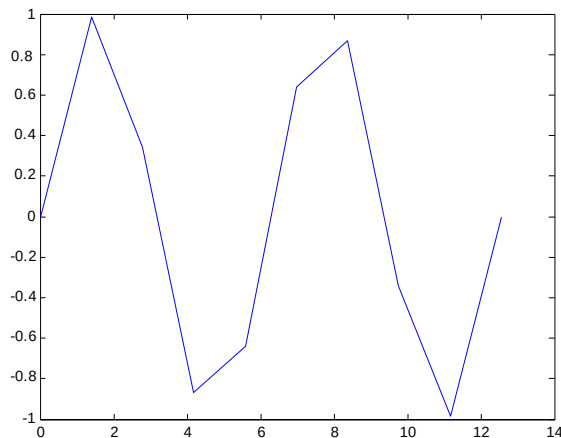
**MATLAB makes visualizing data
fun and easy!**



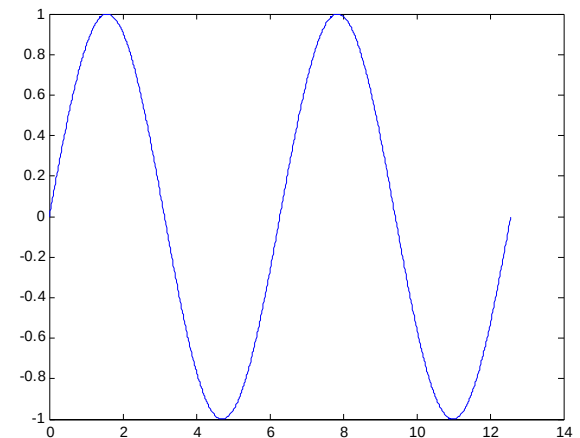
What does plot do?

- **plot** generates dots at each (x,y) pair and then connects the dots with a line
- To make plot of a function look smoother, evaluate at more points
 - » `x=linspace(0,4*pi,1000);`
 - » `plot(x,sin(x));`
- x and y vectors must be same size or else you'll get an error
 - » `plot([1 2], [1 2 3])`
 - error!!

10 x values:



1000 x values:



End of Part 1

- (1) Getting Started
- (2) Scripts
- (3) Making Variables
- (4) Manipulating Variables
- (5) Basic Plotting

Hope that wasn't too much!!

